

README: Project 2 – Building a Simple Web Server

Student ID: 2021-18641

Name: Hadong Lee

1. Implementation Strategy

This project implements a simple web server using non-blocking I/O and the epoll event loop. The code is organized into several modules, each clearly marked with comments in the source code. Below is an overview of the roles and functions of each module:

1.1 HTTP Parser

This module is responsible for parsing incoming HTTP requests. It includes functions such as:

- **parse_first_line**: Validates the initial request line, extracts the HTTP method, URL, and version, and determines the connection type (keep-alive or close).
- **parse_host**: Parses the Host header to ensure that it exists and is correctly formatted.
- **parse_connection**: Processes the Connection header to correctly set the connection persistence.
- **parse_request**: Integrates the parsing of all headers and returns a status code (OK, 400, 403, 404, 500) based on the validity of the request.

1.2 Epoll Context Management

This module manages client connection contexts using a dedicated structure (**connection_context**) that stores:

- The file descriptor for the connection.
- Buffers for reading HTTP requests and constructing response headers.
- File descriptor and state information for serving static files.
- Functions like **create_connection_context**, **reset_context**, and **close_connection** handle resource allocation, context resetting between requests (especially for keep-alive), and proper cleanup upon disconnection.

1.3 HTTP Response Maker

This module builds the HTTP responses to be sent back to the client. It contains:

- **prepare_response**: Prepares a valid HTTP response header by checking file accessibility, setting the **Content-length**, and formatting the header based on the requested file.
- **send_error**: Constructs error responses for various error conditions (400, 403, 404, 500) and ensures that the connection is properly closed after sending the error message.

1.4 Epoll Event Handler

This module implements the event-driven I/O using epoll. It includes:

- **handle_new_connection**: Accepts new client connections, sets them as non-blocking, and adds them to the epoll instance.
- **handle_epollin**: Handles incoming data on a connection by reading the HTTP request, parsing it, and preparing the response.
- **handle_epollout**: Manages sending response headers and body data, using functions like **send_header** and **send_body**. It also handles switching back to the read phase for persistent connections and ensuring proper shutdown during graceful termination.

1.5 Main Function

The **main** function serves as the entry point for the web server. Its responsibilities include:

- Parsing command-line arguments (port and optional root directory).
 - Setting up signal handlers for graceful termination (handling SIGINT and SIGTERM).
 - Initializing the listening socket, binding, and setting it to non-blocking mode.
 - Creating an epoll instance and entering the main event loop where incoming events are handled.
 - Ensuring that active connections are processed before the server shuts down upon receiving a termination signal.
-

2. Program Test

Below is the testing procedure we used to confirm correctness and robustness:

1. Basic Testing with **curl**

- Requested various files to confirm normal data transfer and correct **Content-length**.
- Tested missing or forbidden files (403, 404) during the same runs.
- Verified large file transfers (>2GB) using **curl** and checksums.

2. Malformed Request Testing

- Employed a custom client that sends incomplete or invalid headers, ensuring the server responds with **400 Bad Request**.
- Checked that the parser rejects missing **Host:** or invalid request lines properly.

3. Keep-Alive Tests

- Used a keep-alive test client to send multiple requests over one TCP connection.
- Confirmed that the server reuses the socket for subsequent requests until it sees **Connection: close** or encounters an error.

4. Concurrent Tests with **flexiclient**

- Used the **-active** and **-time** options to open many simultaneous connections.
- Verified the server remains responsive under concurrent I/O.

5. Graceful Termination Test

- Tested graceful termination by requesting a 4GB file transfer and then interrupting the program. Verified that the file was transmitted completely despite the interrupt.
-

3. Known Bugs / Weaknesses

- No confirmed bugs have been identified so far.
 - Potential weaknesses include the need for further security review on input validation and file path sanitization to prevent path traversal attacks.
 - Under extreme concurrent connection scenarios or very large file transfers, additional error handling and resource management optimizations might be required.
-

4. Reference

- epoll.c example: <https://github.com/onestraw/epoll-example/blob/master/epoll.c>